ShipFast — Harden and Sign the Image

Week 3, Session 3 assignment · Advanced Track Lab A3 · Mohammed Awwad

REPOSITORY	RELEASE	SIGNED IMAGE	SCANNED
github.com/MohammedAwwad2/shipfast-devops	distroless (tag v1.0.1)	sha256:3ef62271...0e83	17 Oct 2026

TOOLS

Trivy 0.76.0, Syft 1.54.1, Cosign devel

Every command in this report was run and its output captured on 17 October 2026. Nothing here is illustrative. Vulnerability databases change daily, so all CVE counts are as of that date.

Part A — A distroless image

complete

advanced/docker/Dockerfile.distroless

```
# Build stage: full Node image, needs npm and a shell.
FROM node:24-alpine AS build

WORKDIR /app
ENV NODE_ENV=production

COPY package*.json ./
RUN npm ci --omit=dev && npm cache clean --force

COPY . .

# Runtime stage: distroless. No shell, no package manager, no npm.
# The :nonroot variant already runs as uid 65532 and its ENTRYPOINT is node,
# so CMD carries only the script path, not the word "node".
FROM gcr.io/distroless/nodejs24-debian12:nonroot

WORKDIR /app
ENV NODE_ENV=production

COPY --from=build --chown=nonroot:nonroot /app /app

EXPOSE 3000

ARG REPO_URL
LABEL org.opencontainers.image.source=${REPO_URL}

# No HEALTHCHECK here on purpose: distroless carries no shell and no wget, so
# the Milestone 2 health check cannot run. Health is checked from outside the
# container instead (Compose, or a Kubernetes httpGet probe).

CMD ["index.js"]
```

Two details decided the shape of this file, and both were found by inspecting the base image rather than by guessing:

The entrypoint already runs Node. `docker inspect` on the distroless base reports `ENTRYPOINT ["/nodejs/bin/node"]`. Docker appends `CMD` to that, so a Session 2 style `CMD ["node", "index.js"]` executes `node`

`node index.js` and Node then treats the literal word `node` as a module path. The `CMD` carries the script path alone.

No `HEALTHCHECK`. The Session 2 check called `wget --spider`, and distroless ships no `wget` and no shell for the `CMD` form to run in. Adding one would produce a container stuck at `(unhealthy)` forever. Health is checked from outside instead — by Compose, or by a Kubernetes `httpGet` probe, which does not need anything inside the container.

EVIDENCE

build, run, and the shell attempt

```
# port 3000 busy -> using 3100 for this capture

$ docker images shipfast-app
IMAGE                     ID             DISK USAGE   CONTENT SIZE   EXTRA
shipfast-app:1.0.0        82df4c60b56d        256MB         64.5MB
shipfast-app:distroless   2b932dce333c        209MB         53.5MB

$ docker run -d -p 3100:3000 --name shipfast-distroless shipfast-app:distroless
53ab46b2aa17c42d7bd9b08688d8c029c362cec68f4955180b041960128e874f

$ docker ps --filter name=shipfast-distroless
NAMES                STATUS         PORTS
shipfast-distroless   Up 4 seconds   0.0.0.0:3100->3000/tcp, [::]:3100->3000/tcp

$ curl -s localhost:3100/health
{"status":"healthy","service":"shipfast-app","version":"1.0.0","uptime":"4s"}

$ docker logs shipfast-distroless
ShipFast app v1.0.0 running on port 3000

$ docker run --rm -it --entrypoint sh shipfast-app:distroless
docker: Error response from daemon: failed to create task for container: failed to create shim task: OCI
runtime create failed: runc create failed: unable to start container process: error during container init:
exec: "sh": executable file not found in $PATH

Run 'docker run --help' for more information

$ docker inspect gcr.io/distroless/nodejs24-debian12:nonroot --format "{{json .Config.Entrypoint}}"
["/nodejs/bin/node"]

$ docker inspect shipfast-app:distroless --format "{{.Config.User}} | {{json .Config.Cmd}}"
65532 | ["index.js"]
```

What happened when I tried to open a shell: the container never started. Docker failed at `runc create` with `exec: "sh": executable file not found in $PATH`. This is the point of the image rather than a fault — there is no `/bin/sh` to execute, so a command-injection bug in the application has no interpreter to reach for.

> **Kept local, as instructed.** The distroless image was never pushed and never tagged `latest`, so the Milestone 2 image on GHCR is untouched. Port 3000 was occupied by the Lab A2 systemd service during this capture, so the container was published on 3100; the container still listens on 3000 internally.

Part B — SBOM and vulnerability counts

complete

Syft 1.54.1 and Trivy 0.76.0, both scanned 17 October 2026

syft -o table, both images

```
$ syft ghcr.io/mohammedawwad2/shipfast-app:latest -o table
NAME                                VERSION      TYPE
@gar/promise-retry                  1.0.3        npm
@isaacs/fs-minipass                 4.0.1        npm
@isaacs/string-locale-compare       1.1.0        npm
@npmcli/agent                       4.0.2        npm
@npmcli/arborist                    9.9.1        npm
@npmcli/config                      10.12.0      npm
@npmcli/fs                          5.0.0        npm
   ... (truncated)
   total rows: 233

$ syft shipfast-app:distroless -o table
NAME                    VERSION            TYPE
accepts                 1.3.8              npm
array-flatten           1.1.1              npm
base-files              12.4+deb12u13      deb
body-parser             1.20.8             npm
bytes                   3.1.2              npm
call-bind-apply-helpers 1.0.2              npm
call-bound              1.0.4              npm
   ... (truncated)
   total rows: 80
```

exact counts

```
# Counted from the JSON output of the same scans, so the numbers are exact.

                            GHCR Alpine        Distroless
  syft packages                 235                80
      of which npm              215                69
      of which OS (apk / deb)    19                10
  Trivy CRITICAL                  0                 1
  Trivy HIGH                      4                 6
  Trivy all severities           13                70
      MEDIUM / LOW              8 / 1            35 / 27
```

```
trivy --severity CRITICAL,HIGH
```

```
# The 4 HIGH in the Alpine image -- every one inside npm's own
# bundled dependencies, which never execute at runtime:

  HIGH  brace-expansion       CVE-2026-102276  DoS, stack exhaustion
  HIGH  brace-expansion       CVE-2026-102278  DoS, uncontrolled recursion
  HIGH  http-cache-semantics  CVE-2026-93748   info disclosure, max-stale
  HIGH  undici                CVE-2026-19534   DoS, WebSocket subprotocol

# The 1 CRITICAL and 6 HIGH in the distroless image -- all one package:

  CRITICAL  libssl3  CVE-2026-31789  heap overflow, 32-bit X.509   fix 3.0.19
  HIGH      libssl3  CVE-2026-28387  use-after-free -> code exec   fix 3.0.19
  HIGH      libssl3  CVE-2026-28388  NULL deref -> DoS             fix 3.0.19
  HIGH      libssl3  CVE-2026-28389  DoS in CMS processing         fix 3.0.19
  HIGH      libssl3  CVE-2026-28390  NULL deref -> DoS             fix 3.0.19
  HIGH      libssl3  CVE-2026-45447  use-after-free, PKCS7_verify  fix 3.0.20
  HIGH      libssl3  CVE-2026-84782  information disclosure        no fix
```

Why the numbers differ

In one sentence: distroless lists a third of the packages because it drops npm and the BusyBox userland entirely, but it reports more CVEs because the Debian 12 base it keeps ships `libssl3 3.0.18` — four patch releases behind what Debian itself offers — and glibc and OpenSSL have far longer published advisory histories than musl does.

Worth saying out loud, because the headline numbers mislead in both directions. Every one of the Alpine image's HIGH findings lives inside npm's bundled dependencies, and the container runs `node index.js`, so npm never executes and none of them is reachable. On the distroless side, the single CRITICAL is a heap overflow *on 32-bit systems* triggered by parsing a crafted X.509 certificate; the image is `amd64` and the application has no TLS and no certificate handling at all, so that path is never entered either. A rebuild on a freshly pulled base tag did not change the count, which confirms the gap is in the published base image rather than in anything this repository controls.

Part C — Sign and verify with Cosign

verified

The Milestone 2 image on GHCR, signed by digest

`cosign.key` was added to `.gitignore` before the key pair was generated, so the private key was never a candidate for a commit. The `.gitignore` was later widened to cover `*.key`, `*.pem`, `*.p12`, `*.pfx` and ssh keys, while keeping `*.pub` committable.

cosign generate-key-pair / sign

```
$ cosign generate-key-pair
Enter password for private key:
Private key written to cosign.key
Public key written to cosign.pub

$ cosign sign --key cosign.key ghcr.io/mohammedawwad2/shipfast-
app@sha256:3ef622711899a55f9f2cbe15e148942b24e2f60d51f50fd387cb5dd0f25a0e83
Enter password for private key:
tlog entry created with index: 3294996721
Pushing signature to: ghcr.io/mohammedawwad2/shipfast-app
```

cosign verify --key cosign.pub

```
$ cosign verify --key cosign.pub ghcr.io/mohammedawwad2/shipfast-
app@sha256:3ef622711899a55f9f2cbe15e148942b24e2f60d51f50fd387cb5dd0f25a0e83

Verification for ghcr.io/mohammedawwad2/shipfast-app@sha256:3ef622711899... --
The following checks were performed on each of these signatures:
  - The cosign claims were validated
  - Existence of the claims in the transparency log was verified offline
  - The signatures were verified against the specified public key

[{"critical":{"identity":{"docker-reference":"ghcr.io/mohammedawwad2/shipfast-app"},
  "image":{"docker-manifest-
digest":"sha256:3ef622711899a55f9f2cbe15e148942b24e2f60d51f50fd387cb5dd0f25a0e83"},
  "type":"cosign container image signature"},
  "optional":{"Bundle":{"Payload":{"integratedTime":1792240388,"logIndex":3294996721,...}}}}]
```

digest, key hygiene, registry listing

```
$ cosign version
GitVersion:    devel
GoVersion:     go1.25.0
Platform:      linux/amd64

$ docker inspect --format "{{index .RepoDigests 0}}" ghcr.io/mohammedawwad2/shipfast-app:latest
ghcr.io/mohammedawwad2/shipfast-app@sha256:3ef622711899a55f9f2cbe15e148942b24e2f60d51f50fd387cb5dd0f25a0e83

$ ls advanced/docker
Dockerfile.distroless
cosign.key
cosign.pub
hardening-report.md
sbom-alpine.json

$ git check-ignore -v advanced/docker/cosign.key
.gitignore:9:cosign.key advanced/docker/cosign.key

$ git status --short       # cosign.key must not appear
(clean)

$ git ls-files advanced/docker/
advanced/docker/Dockerfile.distroless
advanced/docker/cosign.pub
advanced/docker/hardening-report.md
advanced/docker/sbom-alpine.json

# signature stored as an extra tag in the same package:
$ curl -s .../tags/list
{"name":"mohammedawwad2/shipfast-app","tags":["1.0.0","latest","sha256-
3ef622711899a55f9f2cbe15e148942b24e2f60d51f50fd387cb5dd0f25a0e83.sig"]}
```

Why a digest and not a tag. A tag is a mutable pointer: anyone with write access to the package can move `latest` to different bytes at any time. A digest is a SHA-256 hash of the image manifest, which itself lists the hash of every layer and of the config, so it is derived from the content rather than assigned to it — change one byte anywhere and the digest changes. Signing a tag therefore means "I approve whatever this name points at today", which stops being true the moment the tag moves; signing a digest means "I approve exactly these bytes". This is also why cosign now prints a warning on tag input and states that signing by tag will be removed in a future release.

The signature is stored in the same package as an extra tag, visible in the registry listing above as `sha256-3ef62271….sig`. The image carries two valid signatures from the same key, at Rekor log indexes `3294975228` and `3294996721` — see the stuck-point below for why.

Part D — The hardening report

committed

advanced/docker/hardening-report.md

Written from the assignment template with every blank filled from the measured output above. It covers the comparison table, what the numbers show, the debugging trade-off, and the signing section. The file is committed; this document is its supporting evidence.

Done checklist

8 of 8

- ✓ **`Dockerfile.distroless` builds, and the container answers on `/health`**
  Built from the `app/` context with `-f ../advanced/docker/Dockerfile.distroless`. `{"status":"healthy",…}` returned.
- ✓ **Both images compared on size, with the numbers recorded**
  Alpine 256 MB on disk / 64.5 MB content; distroless 209 MB / 53.5 MB. A 18 % saving on disk, 17 % on content.
- ✓ **SBOM generated with Syft for the Milestone 2 image**
  `advanced/docker/sbom-alpine.json`, Syft 1.54.1 JSON format, 235 artifacts.
- ✓ **CVE counts recorded for both images**
  Alpine 0 CRITICAL / 4 HIGH; distroless 1 CRITICAL / 6 HIGH. Scanned 17 October 2026 with Trivy 0.76.0.
- ✓ **Image signed by digest and the signature verified with `cosign.pub`**
  Signed `@sha256:3ef62271…`, never the tag. Verify reports the signature validated against the public key.
- ✓ **`cosign.key` is in `.gitignore` and does not appear in `git status`**
  `git check-ignore` confirms the rule; the working tree is clean and only `cosign.pub` is tracked. `git rev-list --all` finds the private key in no commit.
- ✓ **`advanced/docker/hardening-report.md` committed**
  Committed with every blank filled from measured output.
- ✓ **Committed and pushed with the required message**
  `feat: harden and sign shipfast image with distroless, sbom and cosign` — commit `817bbbf`, released as **distroless** on tag `v1.0.1`.

Interview drill

2 of 2

One-sentence answer, the mechanism, an example from my own work, then the trade-off.

1. **What is a distroless image, and what is the trade-off of using one?**

   A distroless image contains your application and its language runtime and nothing else — no shell, no package manager, no coreutils. *Mechanism:* it is built as a multi-stage Dockerfile. The first stage uses a full base image because it needs `npm` and a shell to install dependencies; the final stage starts `FROM` a distroless base and copies only the finished application directory across. Nothing from the build stage's toolchain survives into the image. *From my own work:* my Alpine image lists 235 packages and the distroless one lists 80, and `docker run --entrypoint sh` against it fails with `exec: "sh": executable file not found in $PATH` — there is genuinely no shell inside. *Trade-off:* you lose every debugging tool. No `docker exec ... sh`, no `curl`, no `ps`, and the Milestone 2 `HEALTHCHECK` stops working because it was a `wget` call, so health has to be probed from outside the container. You debug from the outside instead — `docker logs`, `docker inspect` for the exit code and resolved command, and the `:debug` variant of the base when you really need a shell. It is a good trade in production, where a container should be replaced rather than poked at, and an awkward one on a laptop while you are still finding out why it will not boot.

2. **What is an SBOM, and why would a client ask for one?**

   An SBOM is a machine-readable inventory of every component inside a build artifact, with versions. *Mechanism:* a tool like Syft walks the image filesystem, reads each package manager's metadata — `package.json` files for npm, the apk or dpkg database for OS packages — and emits a list in a standard format such as SPDX or CycloneDX. Because it is produced from the image rather than from the source, it records what was actually shipped, not what the manifest intended. *From my own work:* `advanced/docker/sbom-alpine.json` lists 235 components for my Milestone 2 image, and it is what showed me that 215 of them are npm packages while only 67 belong to my application — the rest are npm's own bundled dependencies, which is also where every one of my HIGH findings lives. *Why a client asks:* so they can answer "are we affected?" in minutes instead of weeks. When the next Log4Shell lands, grepping an SBOM across every deployed artifact is a query; opening every repository and rebuilding is a project. It also feeds vulnerability scanning and licence compliance, and US federal buyers now require one. *Trade-off:* an SBOM is a snapshot that goes stale the moment the image is rebuilt, and it lists what is present, not what is reachable — which is exactly the trap in my own numbers, where findings sat in code that never runs. It has to be regenerated per build and read with context, or it becomes a long list that creates work rather than removing it.

Where I got stuck

resolved

1. The distroless container exited immediately. The build succeeded, then the container died at once with `Error: Cannot find module '/home/nonroot/node'`. Two causes, neither visible in the build output: the runtime stage copied nothing from the builder, so `/app` was empty; and `CMD ["node", "index.js"]` was being appended to an entrypoint that already is `node`. Following the assignment's own routine — `docker logs` first, then `docker inspect` on the base image — gave both answers. Fixed with `COPY --from=builder /app /app` and `CMD ["index.js"]`.

2. `cosign sign` failed after creating the signature. I passed the digest as a separate argument instead of attaching it to the repository:

the mistake

```
$ cosign sign --key cosign.key ghcr.io/mohammedawwad2/shipfast-app:latest 3ef622711899...
                               |                                        |
                               +-- image #1 ----------------------------+-- image #2

Error: signing [...]: accessing entity: GET
  https://index.docker.io/v2/library/3ef622711899.../manifests/latest:
  UNAUTHORIZED: authentication required
```

cosign accepts several image references, so it read the bare hex string as a second image, found no registry in it, and defaulted to Docker Hub — hence `index.docker.io/v2/library/3ef62271…: UNAUTHORIZED`. The first reference had already been signed and pushed by then, which is why the image now has two valid signatures. The fix was to write one reference, `repo@sha256:digest`. `cosign clean` would remove the spare from the registry, though the transparency-log entries are permanent by design.

3. `cosign verify` reported a missing file. `loading public key: open ghcr.io/…: no such file or directory` — the arguments were swapped. `--key` always takes the key file and the image is positional, so `cosign verify cosign.pub --key <image>` asks cosign to load an image reference as a public key. Correct form: `cosign verify --key cosign.pub <image>`.

Generated 17 October 2026 · Docker 29.8.0, Trivy 0.76.0, Syft 1.54.1, Cosign devel (go1.25.0), Ubuntu 26.04.1 LTS on WSL2. No token, private key or key password appears in this document.